

Due: Friday, March 8 at 11:59 PM PST

- Homework 4 consists of coding assignments and math problems.
- We prefer that you typeset your answers using $\text{\LaTeX}$ or other word processing software. If you haven't yet learned $\text{\LaTeX}$, one of the crown jewels of computer science, now is a good time! Neatly handwritten and scanned solutions will also be accepted.
- In all of the questions, **show your work**, not just the final answer.
- **We will not provide points back with respect to homework submission errors.** This includes, but is not limited to: 1) not assigning pages to problems; 2) not including code in the write-up appendix; 3) not including code in the designated code Gradescope assignment; 4) not including Kaggle scores; 5) submitting code that only partially works; 6). submitting late regrade requests. **Please carefully read and follow the HW submission guidelines/reminders for Pages 1, 2, and 10 of HW 4.**
- **Start early; you can submit models to Kaggle only twice a day!**

Deliverables:

1. Submit your predictions for the test sets to Kaggle as early as possible. Include your Kaggle scores in your write-up. The Kaggle competition for this assignment can be found at
 - WINE: <https://www.kaggle.com/competitions/cs189-hw4-wine-spring-2024/>
2. Write-up: Submit your solution in **PDF** format to “Homework 4 Write-Up” in Gradescope.
 - On the first page of your write-up, please list students with whom you collaborated
 - Start each question on a new page. If there are graphs, include those graphs on the same pages as the question write-up. **DO NOT** put them in an appendix. We need each solution to be self-contained on pages of its own.
 - **Only PDF uploads to Gradescope will be accepted.** You are encouraged use $\text{\LaTeX}$ or Word to typeset your solution. You may also scan a neatly handwritten solution to produce the PDF.
 - **Replicate all your code in an appendix.** Begin code for each coding question in a fresh page. Do not put code from multiple questions in the same page. When you upload this PDF on Gradescope, *make sure* that you assign the relevant pages of your code from appendix to correct questions.
 - While collaboration is encouraged, *everything* in your solution must be your (and only your) creation. Copying the answers or code of another student is strictly forbidden.

Furthermore, all external material (i.e., *anything* outside lectures and assigned readings, including figures and pictures) should be cited properly. We wish to remind you that consequences of academic misconduct are *particularly severe*!

3. Code: Submit your code as a .zip file to “Homework 4 Code”.

- **Set a seed for all pseudo-random numbers generated in your code.** This ensures your results are replicated when readers run your code. For example, you can seed numpy with `np.random.seed(189)`.
- Include a README with your name, student ID, the values of random seed (above) you used, and instructions for running (and compiling, if appropriate) your code.
- Do NOT provide any data files. Supply instructions on how to add data to your code.
- Code requiring exorbitant memory or execution time might not be considered.
- Code submitted here must match that in the PDF Write-up. The Kaggle score will not be accepted if the code provided a) does not compile or b) compiles but does not produce the file submitted to Kaggle.

Notation: In this assignment we use the following conventions.

- Symbol “defined equal to” ($\triangleq$) *defines* the quantity to its left to be the expression to its right and is equivalent to $:=$.
- Scalars are lowercase non-bold: x, u_1, α_i . Matrices are uppercase alphabets: A, B_1, C_i . Vectors (column vectors) are in bold: $\mathbf{x}, \alpha_1, \mathbf{X}, \mathbf{Y}_j$.
- $\|\mathbf{v}\|$ denotes the Euclidean norm (length) of vector $\mathbf{v}$: $\|\mathbf{v}\| \triangleq \sqrt{\mathbf{v} \cdot \mathbf{v}}$. $\|A\|$ denotes the (operator) norm of matrix A , the magnitude of its largest singular value: $\|A\| = \max_{\|\mathbf{v}\|=1} \|A\mathbf{v}\|$.
- $[n] \triangleq \{1, 2, 3, \dots, n\}$. $\mathbf{1}$ and $\mathbf{0}$ denote the vectors with all-ones and all-zeros, respectively.

1 Honor Code

Declare and sign the following statement (Mac Preview, PDF Expert, and FoxIt PDF Reader, among others, have tools to let you sign a PDF file):

“I certify that all solutions are entirely my own and that I have not looked at anyone else’s solution. I have given credit to all external sources I consulted.”

Signature: _____

2 Logistic Regression with Newton's Method

Given examples $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and associated labels $y_1, y_2, \dots, y_n \in \{0, 1\}$, the cost function for *unregularized* logistic regression is

$$J(\mathbf{w}) \triangleq - \sum_{i=1}^n \left(y_i \ln s_i + (1 - y_i) \ln(1 - s_i) \right)$$

where $s_i \triangleq s(\mathbf{x}_i \cdot \mathbf{w})$, $\mathbf{w} \in \mathbb{R}^d$ is a weight vector, and $s(\gamma) \triangleq 1/(1 + e^{-\gamma})$ is the logistic function.

Define the $n \times d$ design matrix X (whose i^{th} row is $\mathbf{x}_i^\top$), the label n -vector $\mathbf{y} \triangleq [y_1 \dots y_n]^\top$, and $\mathbf{s} \triangleq [s_1 \dots s_n]^\top$. For an n -vector $\mathbf{a}$, let $\ln \mathbf{a} \triangleq [\ln a_1 \dots \ln a_n]^\top$. The cost function can be rewritten in vector form as

$$J(\mathbf{w}) = -\mathbf{y} \cdot \ln \mathbf{s} - (\mathbf{1} - \mathbf{y}) \cdot \ln(\mathbf{1} - \mathbf{s}).$$

Further, recall that for a real symmetric matrix $A \in \mathbb{R}^{d \times d}$, there exist U and Λ such that $A = U\Lambda U^\top$ is the eigendecomposition of A . Here Λ is a diagonal matrix with entries $\{\lambda_1, \dots, \lambda_d\}$. An alternative notation is $\Lambda = \text{diag}(\lambda_i)$, where $\text{diag}()$ takes as input the list of diagonal entries, and constructs the corresponding diagonal matrix. This notation is widely used in libraries like `numpy`, and is useful for simplifying some of the expressions when written in matrix-vector form. For example, we can write $\mathbf{s} = \text{diag}(s_i) \mathbf{1}$.

Hint: See page two for notational conventions used here.

*Hint: Recall matrix calculus identities. The elements in **bold** indicate vectors.*

$$\begin{aligned} \nabla_{\mathbf{x}} \alpha \mathbf{y} &= (\nabla_{\mathbf{x}} \alpha) \mathbf{y}^\top + \alpha \nabla_{\mathbf{x}} \mathbf{y} & \nabla_{\mathbf{x}} (\mathbf{y} \cdot \mathbf{z}) &= (\nabla_{\mathbf{x}} \mathbf{y}) \mathbf{z} + (\nabla_{\mathbf{x}} \mathbf{z}) \mathbf{y}; \\ \nabla_{\mathbf{x}} \mathbf{f}(\mathbf{y}) &= (\nabla_{\mathbf{x}} \mathbf{y}) (\nabla_{\mathbf{y}} \mathbf{f}(\mathbf{y})); & \nabla_{\mathbf{x}} g(\mathbf{y}) &= (\nabla_{\mathbf{x}} \mathbf{y}) (\nabla_{\mathbf{y}} g(\mathbf{y})); \end{aligned}$$

and $\nabla_{\mathbf{x}} C \mathbf{y}(\mathbf{x}) = (\nabla_{\mathbf{x}} \mathbf{y}(\mathbf{x})) C^\top$, where C is a constant matrix.

- 1 Derive the gradient $\nabla_{\mathbf{w}} J(\mathbf{w})$ of cost $J(\mathbf{w})$ as a matrix-vector expression. Also derive *all intermediate derivatives* in matrix-vector form. Do NOT specify them (**including the intermediates**) in terms of their individual components (e.g. w_i for vector $\mathbf{w}$).
- 2 Derive the Hessian $\nabla_{\mathbf{w}}^2 J(\mathbf{w})$ for the cost function $J(\mathbf{w})$ as a matrix-vector expression.
- 3 Write the matrix-vector update law for one iteration of Newton's method, substituting the gradient and Hessian of $J(\mathbf{w})$.
- 4 You are given four examples $\mathbf{x}_1 = [0.2 \ 3.1]^\top$, $\mathbf{x}_2 = [1.0 \ 3.0]^\top$, $\mathbf{x}_3 = [-0.2 \ 1.2]^\top$, $\mathbf{x}_4 = [1.0 \ 1.1]^\top$ with labels $y_1 = 1, y_2 = 1, y_3 = 0, y_4 = 0$. These points cannot be separated by a line passing through origin. Hence, as described in lecture, append a 1 to each $\mathbf{x}_{i \in [4]}$ and use a weight vector $\mathbf{w} \in \mathbb{R}^3$ whose last component is the bias term (called α in lecture). Begin with initial weight $\mathbf{w}^{(0)} = [-1 \ 1 \ 0]^\top$. For the following, state only the final answer with four digits after the decimal point. You may use a calculator or write a program to solve for these, but do NOT submit any code for this part.

- (a) State the value of $s^{(0)}$ (the initial value of s).
 (b) State the value of $w^{(1)}$ (the value of w after 1 iteration).
 (c) State the value of $s^{(1)}$ (the value of s after 1 iteration).
 (d) State the value of $w^{(2)}$ (the value of w after 2 iterations).

$$1. \quad J(w) = -y \cdot \ln s - (1-y) \cdot \ln(1-s) \quad s = s(X \cdot w)$$

$$\nabla_w J(w) = -y \cdot \nabla_w \ln s(X \cdot w) - (1-y) \nabla_w \ln(1-s(X \cdot w))$$

$$= -y \cdot \frac{1}{s(X \cdot w)} \cdot \underbrace{s(X \cdot w) (1-s(X \cdot w))}_{s' = s(1-s)} \underbrace{\nabla_w w \cdot X^T}_{\mathbb{1}}$$

$$= -y \frac{1}{1-s(X \cdot w)} (-s(X \cdot w) \cdot (1-s(X \cdot w))) \nabla_w w \cdot X^T$$

$$= -y (1-s) X^T + (1-y) s \cdot X^T$$

$$= (-y + \cancel{y^T s} + s - \cancel{y^T s}) X^T = -X^T (y - s)$$

$$2. \quad \nabla_w^2 J(w) = \nabla_w [-X^T (y - s)] = \nabla_w (X^T s)$$

$$= \nabla_w (s(X \cdot w)) \cdot X$$

$$= s(1-s) \cdot (\nabla_w w) X^T X = X^T \text{diag}[s_i(1-s_i)] \mathbb{1} X$$

$$\equiv X^T \Omega X$$

$$\Omega \equiv \begin{bmatrix} s_1(1-s_1) & & \\ & s_2(1-s_2) & \\ & & \ddots \\ & & & s_n(1-s_n) \end{bmatrix}$$

$$s = \text{diag}(s_i) \cdot \mathbb{1}$$

$$\nabla_y s(y)$$

$$= \nabla_y \text{diag}(s_i(y)) \cdot \mathbb{1}$$

$$= \text{diag}(\nabla_y (s_i(y))) \cdot \mathbb{1}$$

$$= \text{diag}(s_i(1-s_i)) \cdot \mathbb{1}$$

$$\equiv s(1-s)$$

$$\frac{\partial s_i}{\partial y_j} = \begin{cases} s_i(1-s_i) & i=j \\ 0 & i \neq j \end{cases}$$

$$\Rightarrow \frac{\partial s}{\partial y} = \nabla_y(s)$$

$$= \text{diag}(s_i(1-s_i)) \mathbb{1}$$

3. Newton's method

$$w \leftarrow w - [\nabla^2 J(w)]^{-1} \nabla J(w)$$

$$= w - \underbrace{[X^T \text{diag}(s_i(1-s_i)) \mathbb{1} X]^{-1}}_{\Omega} X^T (s - y)$$

$$4. \quad S^{(0)} = \begin{bmatrix} 0.948 \\ 0.881 \\ 0.802 \\ 0.525 \end{bmatrix} \quad W^{(1)} = \begin{bmatrix} 1.325 \\ 3.950 \\ -6.829 \end{bmatrix}$$

$$S^{(1)} = \begin{bmatrix} 0.947 \\ 0.975 \\ 0.031 \\ 0.104 \end{bmatrix} \quad W^{(2)} = \begin{bmatrix} 1.366 \\ 4.160 \\ -9.200 \end{bmatrix}$$

3 Wine Classification with Logistic Regression

The wine dataset `data.mat` consists of 6,000 sample points, each having 12 features. The description of these features is provided in `data.mat`. The dataset includes a training set of 5,000 sample points and a test set of 1,000 sample points. Your classifier needs to predict whether a wine is white (class label 0) or red (class label 1).

Begin by normalizing the data with each feature's mean and standard deviation. You should use training data statistics to normalize both training and validation/test data. Then add a fictitious dimension. Whenever required, it is recommended that you tune hyperparameter values with cross-validation.

Please set a random seed whenever needed and **report it**.

Use of automatic logistic regression libraries/packages is prohibited for this question. If you are coding in python, it is better to use `scipy.special.expit` for evaluating logistic functions as its code is numerically stable, and doesn't produce NaN or `MathOverflow` exceptions.

- 1 *Batch Gradient Descent Update.* State the batch gradient descent update law for logistic regression **with ℓ_2 regularization**. As this is a “batch” algorithm, each iteration should use *every training example*. You don't have to show your derivation. You may reuse results from your solution to question 2.1.
- 2 *Batch Gradient Descent Code.* Implement your batch gradient descent algorithm for logistic regression and include your code here. Choose reasonable values for the regularization parameter and step size (learning rate), specify your chosen values in the write-up, and train your model from question 3.1. Shuffle and split your data into training/validation sets and mention the random seed used in the write-up. Plot the value of the cost function versus the number of iterations spent in training.
- 3 *Stochastic Gradient Descent (SGD) Update.* State the SGD update law for logistic regression with ℓ_2 regularization. Since this is not a “batch” algorithm anymore, each iteration uses *just one* training example. You don't have to show your derivation.
- 4 *Stochastic Gradient Descent Code.* Implement your stochastic gradient descent algorithm for logistic regression and include your code here. Choose a suitable value for the step size (learning rate), specify your chosen value in the write-up, and run your SGD algorithm from question 3.3. Shuffle and split your data into training/validation sets and mention the random seed used in the write-up. Plot the value of the cost function versus the number of iterations spent in training.

Compare your plot here with that of question 3.2. Which method converges more quickly? Briefly describe what you observe.
- 5 Instead of using a constant step size (learning rate) in SGD, you could use a step size that slowly shrinks from iteration to iteration. Run your SGD algorithm from question 3.3 with a step size $\epsilon_t = \delta/t$ where t is the iteration number and δ is a hyperparameter you select

empirically. Mention the value of δ chosen. Plot the value of cost function versus the number of iterations spent in training.

How does this compare to the convergence of your previous SGD code?

- 6 *Kaggle*. Train your *best* classifier on the entire training set and submit your prediction on the test sample points to Kaggle. As always for Kaggle competitions, you are welcome to add or remove features, tweak the algorithm, and do pretty much anything you want to improve your Kaggle leaderboard performance **except** that you may not replace or augment logistic regression with a wholly different learning algorithm. Your code should output the predicted labels in a CSV file.

Report your Kaggle username and your best score, and briefly describe what your best classifier does to achieve that score.

1. Batch Gradient Descent

$$J(w) = -y \ln S(X \cdot w) - (1-y) \ln (1 - S(X \cdot w)) + \lambda \|w\|^2$$

$$\Rightarrow \nabla_w J(w) = -y \frac{S(1-S)}{S} X^T - (1-y) \frac{-(1-S)S}{1-S} X^T + 2\lambda w$$

$$= -X^T (y - S) + 2\lambda w$$

$$w \leftarrow w - \epsilon \cdot \nabla_w J(w)$$

$$= w + \epsilon X^T (y - S) - 2\lambda \epsilon w$$

$$= (1 - 2\lambda \epsilon) w + \epsilon X^T (y - S)$$

3. Stochastic Gradient Descent

Each time we only use x_i, y_i

here $s_i = S(x_i \cdot w)$

$$\Rightarrow w \leftarrow w + \epsilon (y_i - s_i) X_i^T - 2\lambda \epsilon w$$

$$= (1 - 2\lambda \epsilon) w + \epsilon (y_i - s_i) X_i^T$$

4 A Bayesian Interpretation of Lasso

Suppose you are aware that the labels $y_{i \in [n]}$ corresponding to sample points $\mathbf{x}_{i \in [n]} \in \mathbb{R}^d$ follow the density law

$$f(y_i | \mathbf{x}_i, \mathbf{w}) = \frac{1}{\sigma \sqrt{2\pi}} e^{-(y_i - \mathbf{w} \cdot \mathbf{x}_i)^2 / (2\sigma^2)}$$

where $\sigma > 0$ is a known constant and $\mathbf{w} \in \mathbb{R}^d$ is a random parameter. Suppose further that experts have told you that

- each component of $\mathbf{w}$ is independent of the others, and
- each component of $\mathbf{w}$ has the Laplace distribution with location 0 and scale being a known constant b . That is, each component w_i obeys the density law $f(w_i) = e^{-|w_i|/b} / (2b)$.

Assume the outputs $y_{i \in [n]}$ are independent from each other.

Your goal is to find the choice of parameter $\mathbf{w}$ that is *most likely* given the input-output examples $(\mathbf{x}_i, y_i)_{i \in [n]}$. This method of estimating parameters is called *maximum a posteriori* (MAP); Latin for “maximum [odds] from what follows.”

1. Derive the *posterior* probability density law $f(\mathbf{w} | (\mathbf{x}_i, y_i)_{i \in [n]})$ for $\mathbf{w}$ up to a *proportionality constant* by applying Bayes' Theorem and substituting for the densities $f(y_i | \mathbf{x}_i, \mathbf{w})$ and $f(\mathbf{w})$. Don't try to derive an exact expression for $f(\mathbf{w} | (\mathbf{x}_i, y_i)_{i \in [n]})$, as the denominator is very involved and irrelevant to maximum likelihood estimation.
2. Define the log-likelihood for MAP as $\ell(\mathbf{w}) \triangleq \ln f(\mathbf{w} | (\mathbf{x}_i, y_i)_{i \in [n]})$. Show that maximizing the MAP log-likelihood over all choices of $\mathbf{w}$ is the same as minimizing $\sum_{i=1}^n (y_i - \mathbf{w} \cdot \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|_1$ where $\|\mathbf{w}\|_1 = \sum_{j=1}^d |w_j|$ and λ is a constant. Also give a formula for λ as a function of the distribution parameters.

$$1. \quad f(\mathbf{w} | (\mathbf{x}_i, y_i)_{i \in [n]}) = \frac{f(y_i | \mathbf{x}_i, \mathbf{w})_{i \in [n]} f(\mathbf{w})}{f(y_i | \mathbf{x}_i)_{i \in [n]}} \quad \left| \begin{array}{l} \text{Bayes:} \\ f(\mathbf{w} | \text{data}) = \frac{f(\text{data} | \mathbf{w}) \cdot f(\mathbf{w})}{f(\text{data})} \end{array} \right.$$

$$f(y_i | \mathbf{x}_i, \mathbf{w}) = \frac{1}{\sigma \sqrt{2\pi}} \exp(- (y_i - \mathbf{w} \cdot \mathbf{x}_i)^2 / 2\sigma^2)$$

$$f(\mathbf{w} | \mathbf{x}_i) = \exp(-|w_i|/b) / 2b$$

$$\Rightarrow f(\mathbf{w} | (\mathbf{x}_i, y_i)_{i \in [n]}) = \frac{1}{f(y_i)} \left(\frac{1}{\sigma \sqrt{2\pi}} \right)^n \frac{1}{2b} \exp(-\sum_i (y_i - \mathbf{w} \cdot \mathbf{x}_i)^2 / 2\sigma^2 - \sum_i |w_i|/b)$$

$$2. \quad \log f \sim -\sum_i (y_i - \mathbf{w} \cdot \mathbf{x}_i)^2 / 2\sigma^2 - \sum_i |w_i| / b$$

maximize MAP is the same as

$$\text{minimizing } \sum_i (y_i - \mathbf{w} \cdot \mathbf{x}_i)^2 / 2\sigma^2 + \sum_i |w_i| / b$$

$$\text{let } \lambda \equiv 2\sigma^2 / b \Rightarrow \text{minimizing } (y_i - \mathbf{w} \cdot \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|_1$$

5 ℓ_1 -regularization, ℓ_2 -regularization, and Sparsity

You are given a design matrix X (whose i^{th} row is sample point $\mathbf{x}_i^T$) and an n -vector of labels $\mathbf{y} \triangleq [y_1 \dots y_n]^T$. For simplicity, assume X is whitened, so $X^T X = nI$. Do not add a fictitious dimension/bias term; for input $\mathbf{0}$, the output is always 0. Let $\mathbf{x}_{*i}$ denote the i^{th} column of X .

1. The ℓ_p -norm for $w \in \mathbb{R}^d$ is defined as $\|w\|_p = (\sum_{i=1}^d |w_i|^p)^{1/p}$, where $p > 0$. Plot the isocontours with $w \in \mathbb{R}^2$, for the following norms.

(a) $\ell_{0.5}$ (b) ℓ_1 (c) ℓ_2

Use of automatic libraries/packages for computing norms is prohibited for the question.

2. Show that the cost function for ℓ_1 -regularized least squares, $J_1(\mathbf{w}) \triangleq \|X\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|_1$ (where $\lambda > 0$), can be rewritten as $J_1(\mathbf{w}) = \|\mathbf{y}\|^2 + \sum_{i=1}^d f(\mathbf{x}_{*i}, \mathbf{w}_i)$ where $f(\cdot, \cdot)$ is a suitable function whose first argument is a vector and second argument is a scalar.
3. Using your solution to part 2 derive necessary and sufficient conditions for the i^{th} component of the optimizer $\mathbf{w}^*$ of $J_1(\cdot)$ to satisfy each of these three properties: $w_i^* > 0$, $w_i^* = 0$, and $w_i^* < 0$.
4. For the optimizer $\mathbf{w}^\#$ of the ℓ_2 -regularized least squares cost function $J_2(\mathbf{w}) \triangleq \|X\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|^2$ (where $\lambda > 0$), derive a necessary and sufficient condition for $\mathbf{w}_i^\# = 0$, where $\mathbf{w}_i^\#$ is the i^{th} component of $\mathbf{w}^\#$.
5. A vector is called *sparse* if most of its components are 0. From your solution to part 3 and 4, which of $\mathbf{w}^*$ and $\mathbf{w}^\#$ is more likely to be sparse? Why?

$$\begin{aligned}
 2. \quad J_1(\mathbf{w}) &= \|X\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|_1 \\
 &= (X\mathbf{w} - \mathbf{y})^T (X\mathbf{w} - \mathbf{y}) + \lambda \sum_{i=1}^d |w_i| \\
 &= (\mathbf{w}^T X^T - \mathbf{y}^T) (X\mathbf{w} - \mathbf{y}) + \lambda \sum_{i=1}^d |w_i| \\
 &= \mathbf{w}^T X^T X \mathbf{w} - 2 \mathbf{y}^T X \mathbf{w} + \mathbf{y}^T \mathbf{y} + \lambda \sum_{i=1}^d |w_i| \\
 &= \|\mathbf{y}\|^2 + \underbrace{\mathbf{w}^T X^T X \mathbf{w}}_{nI} - 2 \mathbf{y}^T X \mathbf{w} + \lambda \sum_{i=1}^d |w_i| \\
 &= \|\mathbf{y}\|^2 + n \mathbf{w}^T \mathbf{w} - 2 \mathbf{y}^T X \mathbf{w} + \lambda \sum_{i=1}^d |w_i| \\
 &= \|\mathbf{y}\|^2 + n \sum_{i=1}^d w_i^2 - 2 \mathbf{y}^T X \mathbf{w} + \lambda \sum_{i=1}^d |w_i|
 \end{aligned}$$

$$X\mathbf{w} = \sum_{i=1}^d X_{*i} w_i \Rightarrow 2 \mathbf{y}^T X \mathbf{w} = 2 \mathbf{y}^T \left(\sum_{i=1}^d X_{*i} w_i \right)$$

$$\Rightarrow J_1(\mathbf{w}) = \|\mathbf{y}\|^2 + \sum_{i=1}^d \underbrace{-2 \mathbf{y}^T X_{*i} w_i + n w_i^2 + \lambda |w_i|}_{f(X_{*i}, w_i)}$$

3. $\nabla_w J_1(w) = 0$

$$\frac{\partial J_1(w)}{\partial w_i} = -2y^T \cdot X_{*i} + 2n w_i + \lambda \cdot \text{sgn}(w_i)$$

$$w_i = 0 \Rightarrow -2y^T \cdot X_{*i} + \lambda [-1, 1] = 0 \Rightarrow -\frac{\lambda}{2} \leq y^T \cdot X_{*i} \leq \frac{\lambda}{2}$$

$$w_i < 0 \Rightarrow -2y^T \cdot X_{*i} + 2n w_i - \lambda = 0$$

$$\Rightarrow w_i^* = \frac{2y^T \cdot X_{*i} + \lambda}{2n} < 0 \Rightarrow y^T \cdot X_{*i} < -\frac{\lambda}{2}$$

$$w_i > 0 \Rightarrow -2y^T \cdot X_{*i} + 2n w_i + \lambda = 0$$

$$\Rightarrow w_i^* = \frac{2y^T \cdot X_{*i} - \lambda}{2n} > 0 \Rightarrow y^T \cdot X_{*i} > \frac{\lambda}{2}$$

$$\Rightarrow w^* = \frac{2y^T X - \lambda \text{diag}(\text{sgn}(w_i)) \mathbf{1}}{2n}$$

$$\text{sgn} = \begin{cases} 1 & w_i > 0 \\ 0 & w_i = 0 \\ -1 & w_i < 0 \end{cases}$$

4. $\nabla_w J_2(w)$

$$J_2(w) = \|y\|^2 + n w^T w - 2y^T X w + \lambda \sum_{i=1}^d w_i^2$$

$$= \|y\|^2 + \sum_{i=1}^n (n + \lambda) w_i^2 - 2y^T X_{*i} w_i$$

$$\frac{\partial J_2(w)}{\partial w_i} = 2(n + \lambda) w_i - 2y^T X_{*i} = 0$$

$$\Rightarrow w_i^{\#} = \frac{y^T X_{*i}}{(n + \lambda)}$$

$$\text{if } w_i^{\#} = 0 \Rightarrow y^T X_{*i} = 0$$

5. w_i^* since $w_i^{\#} = 0$ only when $y^T X_{*i} = 0$

but $w_i^* = 0$ when $-\frac{\lambda}{2} \leq y^T X_{*i} \leq \frac{\lambda}{2}$

Submission Checklist

Please ensure you have completed the following before your final submission.

At the beginning of your writeup...

1. Have you copied and hand-signed the honor code specified in Question 1?
2. Have you listed all students (Names and ID numbers) that you collaborated with?

In your writeup for Question 3...

1. Have you included your **Kaggle Score** and **Kaggle Username**?

At the end of the writeup...

1. Have you provided a code appendix including all code you wrote in solving the homework?

Executable Code Submission

1. Have you created an archive containing all “.py” files that you wrote or modified to generate your homework solutions?
2. Have you removed all data and extraneous files from the archive?
3. Have you included a README file in your archive containing any special instructions to reproduce your results?

Submissions

1. Have you submitted your written solutions to the Gradescope assignment titled **HW4 Write-Up** and selected pages appropriately?
2. Have you submitted your executable code archive to the Gradescope assignment titled **HW4 Code**?
3. Have you submitted your test set predictions for **Wine** dataset to the appropriate Kaggle challenge?

Congratulations! You have completed Homework 4.